



AIR UNIVERSITY

Department of Computer Science



Operating Systems Lab Project

Project Title:

Kubernetes Cluster as a Distributed Operating Environment



Submitted To:

Ma'am Qurt-ul-Ain



Submitted By:

Muhammad Hamza Naseer | 247121

Sultan Majeed | 247187

Muhammad Arqam | 247107

Abdullah Warraich | 247338

Aryan Miran | 247117

Masroor Ahmed | 247158

Hamza Kaleem | 247151



Semester:

Spring 2026



Course:

Operating Systems Lab



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: os-lab-project
spec:
  replicas: 3
  ...
```



Project Title: Kubernetes Cluster as a Distributed Operating Environment

1. Abstract

This project shows how Operating System concepts can be applied practically using Kubernetes. In this project, we will create a Kubernetes cluster with one control plane node and two worker nodes. All nodes will run on Ubuntu Server 24.04.4 LTS using VMware Workstation.

Kubernetes works like a distributed operating environment. It manages containers, resources, networking, security, and application workloads across multiple machines. This is similar to how an Operating System manages processes, memory, CPU, users, and networking on one computer.

We will deploy a custom full-stack web application with a MySQL database on the cluster.. This will help us demonstrate pod scheduling, load balancing, scaling, self-healing, and application management. We will also use Kubernetes Dashboard, RBAC security, CNI networking, Services, Ingress, resource limits, and health checks.

This project will help us connect OS theory with real practical system administration and container orchestration.

2. Objectives

The objectives of this project are:

1. To understand Operating System concepts through Kubernetes.
2. To create a multi-node Kubernetes cluster.
3. To use one control plane node and two worker nodes.
4. To configure Kubernetes networking using a CNI plugin.
5. To containerize a web application using Docker.
6. To deploy the application using Kubernetes Deployment and Service.
7. To show load balancing between multiple pod replicas.
8. To configure Ingress for external access to the application.
9. To deploy Kubernetes Dashboard for cluster monitoring.
10. To secure Dashboard access using RBAC.
11. To demonstrate scaling, self-healing, resource limits, and pod scheduling.





3. OS Concepts Covered

OS Concept	Kubernetes Implementation
Process Management	Pods and containers are managed like processes.
CPU Scheduling	Kubernetes decides which node will run each pod.
Memory Management	Memory requests and limits control container memory usage.
CPU Management	CPU requests and limits control processing power.
Networking	CNI, Services, and Ingress handle communication.
Access Control	RBAC and ServiceAccounts control permissions.
Fault Tolerance	Failed or deleted pods are recreated automatically.
Load Balancing	Services distribute traffic between pod replicas.
Monitoring	Dashboard shows cluster health and resources.
Distributed Management	Control plane manages multiple worker nodes.
Application Management	Deployments manage replicas, updates, and rollback.

4. System Requirements

Host Machine Requirements

Component	Specification
RAM	16 GB
Free Storage	Around 80 GB
Processor	4 cores or higher recommended
Virtualization	Enabled
Host OS	Windows 10/11
Virtualization Software	VMware Workstation
Internet	Required for downloading packages and images

Virtual Machine Requirements

VM Name	Role	RAM	CPU	Storage
k8-master	Control Plane Node	4 GB	2 cores	25 GB
k8-worker1	Worker Node 1	2 GB	2 cores	18 GB
k8-worker2	Worker Node 2	2 GB	2 cores	18 GB





Software Requirements

Software / Tool	Purpose
Ubuntu Server 24.04.4 LTS	Operating system for Kubernetes nodes
VMware Workstation	To create virtual machines
containerd	Container runtime
kubeadm	To create and join Kubernetes cluster
kubelet	Runs Kubernetes services on each node
kubectl	Command-line tool for managing Kubernetes
Calico / Flannel	CNI plugin for pod networking
Docker	To build application image
Kubernetes Dashboard	To monitor cluster resources
RBAC	To secure access
NGINX Ingress Controller	To route external traffic
Node.js / Express.js / MySQL	To create the custom full-stack web application

Network Requirements

Item	Configuration
VMware Network Mode	NAT
IP Type	Static IP
Network Interface	ens33
Subnet	192.168.193.0/24
Gateway	192.168.193.2

Planned Node IPs

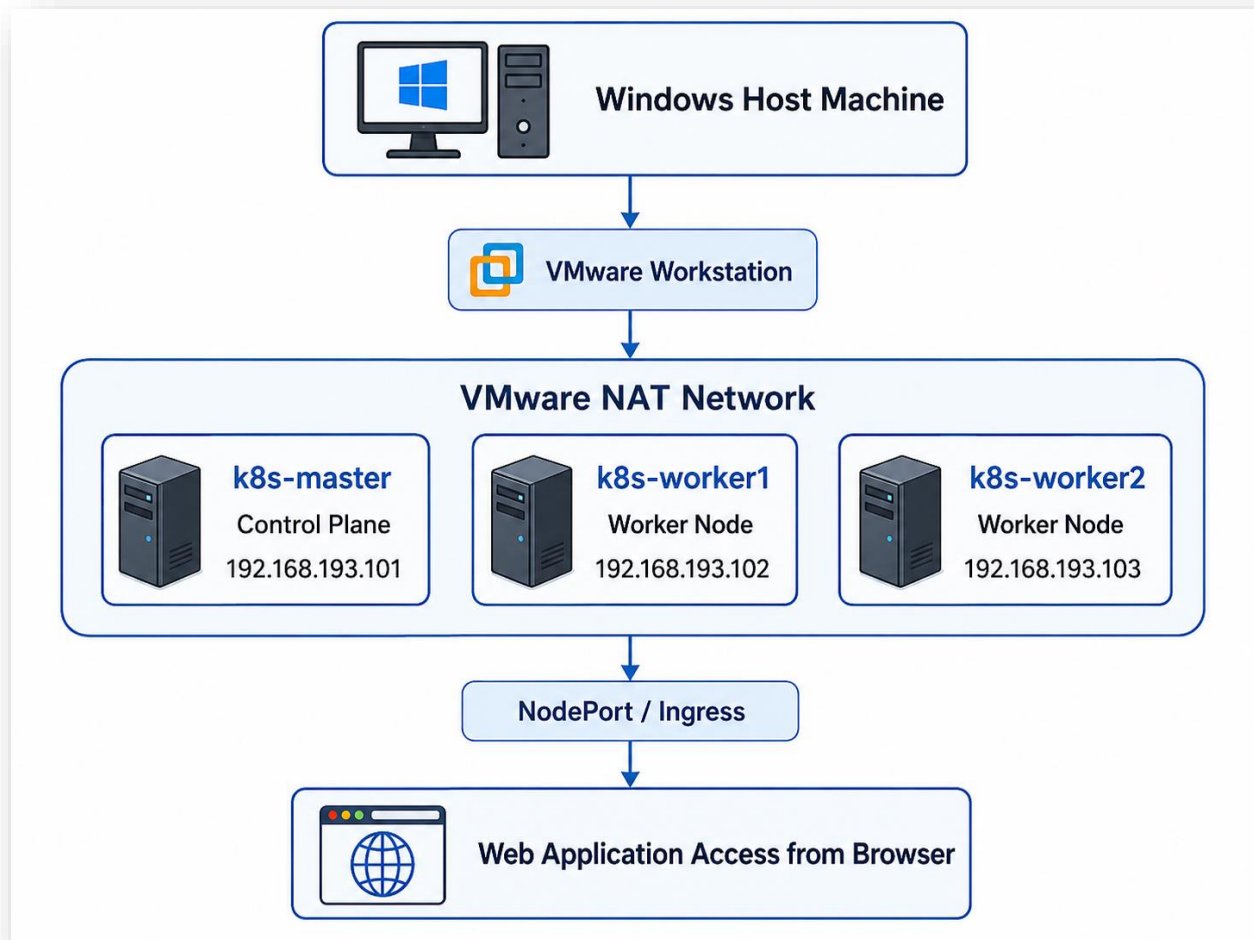
Node	Role	Static IP
k8-master	Control Plane	192.168.193.101
k8-worker1	Worker Node	192.168.193.102
k8-worker2	Worker Node	192.168.193.103

5. Project Architecture

The project is based on a multi-node Kubernetes cluster. It includes one control plane node and two worker nodes. The control plane manages the cluster, while the worker nodes run the application pods.

All virtual machines are connected through VMware NAT networking with static IP addresses. A containerized web application will be deployed on the worker nodes using Kubernetes Deployment, Service, and Ingress.





6. Implementation Steps:

Phase 1: Static IP and Hostname Configuration

Static IP addresses were configured on all Kubernetes nodes to ensure stable communication between the control plane and worker nodes. VMware NAT networking was used so the cluster could remain independent from home or university Wi-Fi networks.

Each node was also assigned a unique hostname to identify its role in the cluster.

Node	Role	Hostname	Static IP
Master Node	Control Plane	k8-master	192.168.193.101
Worker Node 1	Worker	k8-worker1	192.168.193.102
Worker Node 2	Worker	k8-worker2	192.168.193.103

The `/etc/hosts` file was updated on all nodes so they could communicate using hostnames instead of only IP addresses.



SSH Access Configuration

SSH access was used to manage the Ubuntu Server virtual machines from the host machine terminal. This made command execution, configuration, and troubleshooting easier during the cluster setup.

Phase 2: Common Kubernetes Requirements

Before installing Kubernetes, basic system requirements were configured on all Ubuntu nodes. System packages were updated, swap memory was disabled, and required kernel modules were enabled.

Swap was disabled because Kubernetes needs direct control over memory management. Kernel modules such as `overlay` and `br_netfilter` were enabled to support container networking and packet filtering.

Network forwarding settings were also configured so that pods and services could communicate properly inside the Kubernetes cluster.

```
ghost@k8-master:~$ lsmod | grep overlay
overlay                212992  0
ghost@k8-master:~$ lsmod | grep br_netfilter
br_netfilter           32768  0
bridge                 425984  1 br_netfilter
```

Phase 3: Container Runtime Installation

`containerd` was installed as the container runtime for Kubernetes. A container runtime is responsible for running containers inside pods. The default configuration file was generated, and `SystemdCgroup` was enabled to make container resource management compatible with Kubernetes.

After installation, the `containerd` service was restarted and enabled so it can start automatically when the system boots.

```
ghost@k8-master:~$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-17 07:52:55 UTC; 20s ago
     Docs: https://containerd.io
   Main PID: 11130 (containerd)
    Tasks: 7
   Memory: 13.6M (peak: 16.7M)
      CPU: 171ms
   CGroup: /system.slice/containerd.service
           └─11130 /usr/bin/containerd
```





Phase 4: Kubernetes Tools Installation

Kubernetes tools were installed on all nodes. `kubeadm` was used to initialize the cluster and join worker nodes. `kubelet` was installed as the node agent that manages pods on each machine.

`kubectl` was installed as the command-line tool used to control and manage the Kubernetes cluster.

The official Kubernetes package repository was added, and the installed packages were held to avoid accidental version changes during system updates.

```
ghost@k8-master:~$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.8", GitCommit:"1f328c5e9dd683d0c5e69f3d7d58f8371278dec2", GitTreeState:"clean", BuildDate:"2026-05-12T09:52:10Z", GoVersion:"go1.25.9", Compiler:"gc", Platform:"linux/amd64"}
ghost@k8-master:~$ kubectl version --client
Client Version: v1.34.8
Kustomize Version: v5.7.1
ghost@k8-master:~$ kubelet --version
Kubernetes v1.34.8
```

Phase 5: Kubernetes Cluster Initialization

The Kubernetes control plane was initialized on the master node using `kubeadm init`. The control plane components include the API Server, Scheduler, Controller Manager, and etcd database.

After initialization, the Kubernetes configuration file was copied to the user directory so that `kubectl` could manage the cluster. The master node was then verified using `kubectl get nodes`, and control plane pods were checked inside the `kube-system` namespace.

At this stage, the master node appeared as `NotReady` because the CNI network plugin had not been installed yet.

```
ghost@k8-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE    VERSION
k8-master     NotReady  <none>   4m35s  v1.34.8
ghost@k8-master:~$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
etcd-k8-master                      1/1     Running   1           5m39s
kube-apiserver-k8-master             1/1     Running   1           5m46s
kube-controller-manager-k8-master   1/1     Running   1           5m42s
kube-scheduler-k8-master             1/1     Running   1           5m45s
```





Phase 6: Worker Nodes Joining

After initializing the Kubernetes control plane, the worker nodes were joined to the cluster using the `kubeadm join` command generated by the master node. This command connected both worker nodes to the API server running on the control plane.

After joining, the cluster nodes were verified using the `kubectl get nodes` command from the master node. At this stage, all nodes appeared as `NotReady` because the CNI network plugin had not been installed yet.

```
ghost@k8-master:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8-master	NotReady	control-plane	16m	v1.34.8
k8-worker1	NotReady	<none>	3m56s	v1.34.8
k8-worker2	NotReady	<none>	3m49s	v1.34.8

Phase 7: CNI Network Plugin Installation

A Container Network Interface plugin was installed to enable networking inside the Kubernetes cluster. Calico was used as the CNI plugin. It allows pods running on different nodes to communicate with each other.

Before installing the CNI plugin, the nodes were shown as `NotReady`. After the CNI installation, the nodes changed to `Ready`, which confirmed that cluster networking was working properly.

```
ghost@k8-master:~$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
calico-kube-controllers-6b6d455f6d-hcrwx	0/1	ContainerCreating	0	2m13s
calico-node-c8zgv	0/1	Init:2/3	0	2m13s
calico-node-jvgxk	1/1	Running	0	2m13s
calico-node-m7rht	0/1	Init:2/3	0	2m13s
coredns-66bc5c9577-6mb9v	0/1	ContainerCreating	0	12m
coredns-66bc5c9577-pcjsd	0/1	ContainerCreating	0	12m
etcd-k8-master	1/1	Running	1	20m
kube-apiserver-k8-master	1/1	Running	1	20m
kube-controller-manager-k8-master	1/1	Running	1	20m
kube-proxy-c2ltd	1/1	Running	0	8m10s
kube-proxy-jv6sl	1/1	Running	0	12m
kube-proxy-wqm78	1/1	Running	0	8m3s
kube-scheduler-k8-master	1/1	Running	1	20m





Phase 8: Application Deployment

A separate namespace named `os-lab` was created for the project application. A Kubernetes Deployment was then applied to run the web application with three replicas.

The Deployment ensures that the desired number of pods remains running. If any pod fails or is deleted, Kubernetes automatically creates a replacement pod. Resource requests and limits were also added to control CPU and memory usage. A basic NGINX deployment was first used to test Kubernetes deployment and service functionality before deploying the final custom full-stack Node.js and MySQL application.

```
ghost@k8-master:~$ kubectl get deployments -n os-lab
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
os-k8-app     3/3     3            3           2m24s
ghost@k8-master:~$ kubectl get pods -n os-lab -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE          NOMINATED NODE   READINESS GATES
os-k8-app-79fdfcc99b-bpx5l          1/1     Running   0           2m40s  172.16.86.132   k8-worker2    <none>            <none>
os-k8-app-79fdfcc99b-gbcz5          1/1     Running   0           2m40s  172.16.80.130   k8-worker1    <none>            <none>
os-k8-app-79fdfcc99b-klw6p          1/1     Running   0           2m40s  172.16.80.129   k8-worker1    <none>            <none>
```

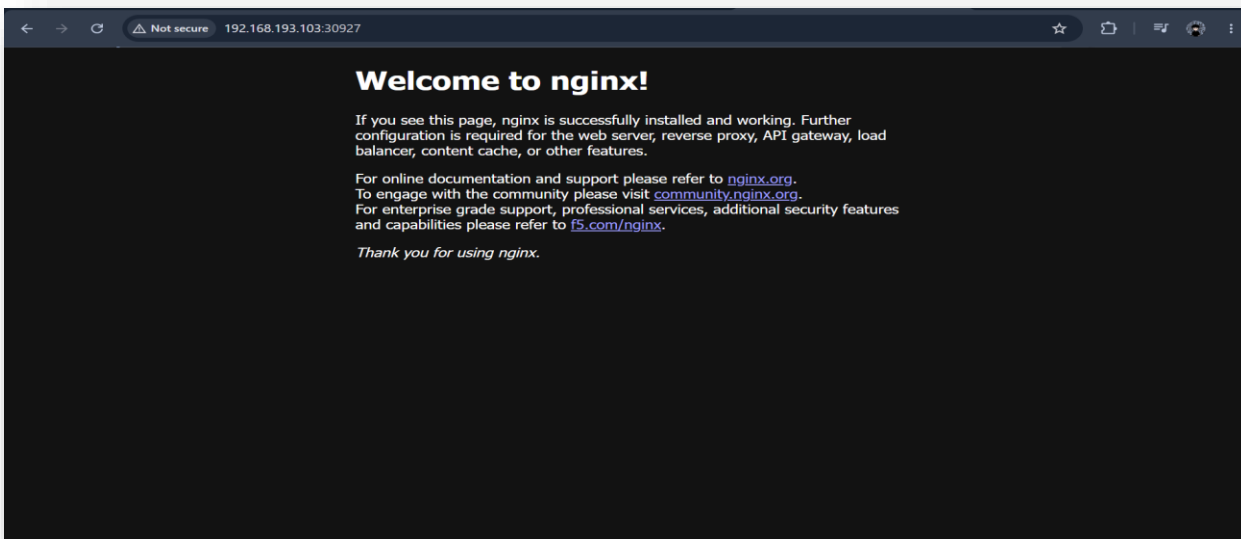
Phase 9: Service and Load Balancing

A Kubernetes Service was created to expose the deployed web application. The Service type was set to `NodePort`, which allows the application to be accessed from outside the cluster using a worker node IP address and the assigned port.

In this project, the service exposed the application on port `30927`. The application was accessed from the browser using the worker node IP and `NodePort`.

The Service also provides load balancing by forwarding traffic to available pod replicas.





Phase 10: Scaling and Self-Healing Demonstration

Kubernetes scaling was tested by increasing the number of application replicas from 3 to 5. After scaling, Kubernetes scheduled new pods across the available worker nodes.

Self-healing was also tested by manually deleting one running pod. Kubernetes automatically created a new pod to maintain the desired number of replicas. This shows how Kubernetes handles fault tolerance and application recovery.

```
ghost@k8-master:~$ kubectl scale deployment os-k8-app --replicas=5 -n os-lab
deployment.apps/os-k8-app scaled
ghost@k8-master:~$ kubectl get pods -n os-lab -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
os-k8-app-79fdfcc99b-bpx5l	1/1	Running	0	12m	172.16.86.132	k8-worker2	<none>		<none>	
os-k8-app-79fdfcc99b-gbcz5	1/1	Running	0	12m	172.16.80.130	k8-worker1	<none>		<none>	
os-k8-app-79fdfcc99b-klw6p	1/1	Running	0	12m	172.16.80.129	k8-worker1	<none>		<none>	
os-k8-app-79fdfcc99b-vdv67	1/1	Running	0	7s	172.16.86.133	k8-worker2	<none>		<none>	
os-k8-app-79fdfcc99b-wg4w5	1/1	Running	0	7s	172.16.80.131	k8-worker1	<none>		<none>	

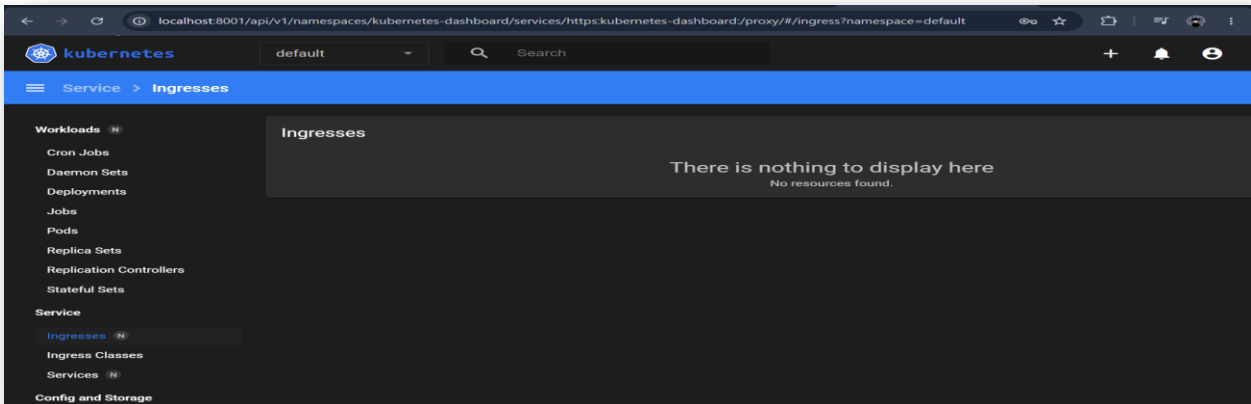
Phase 11: Kubernetes Dashboard and RBAC Configuration

Kubernetes Dashboard was deployed to provide a web-based interface for monitoring cluster resources. It allows us to view nodes, pods, deployments, services, namespaces, and workload status in a simple graphical interface.

For secure access, a custom ServiceAccount was created for the Dashboard. RBAC was configured to give this account limited read-only permissions instead of full administrator access. This follows the principle of least privilege, where a user only gets the permissions required for their task.



The Dashboard was accessed securely using SSH port forwarding and `kubectl proxy`. This allowed localhost-based access from the host machine without exposing the Dashboard directly on the network.

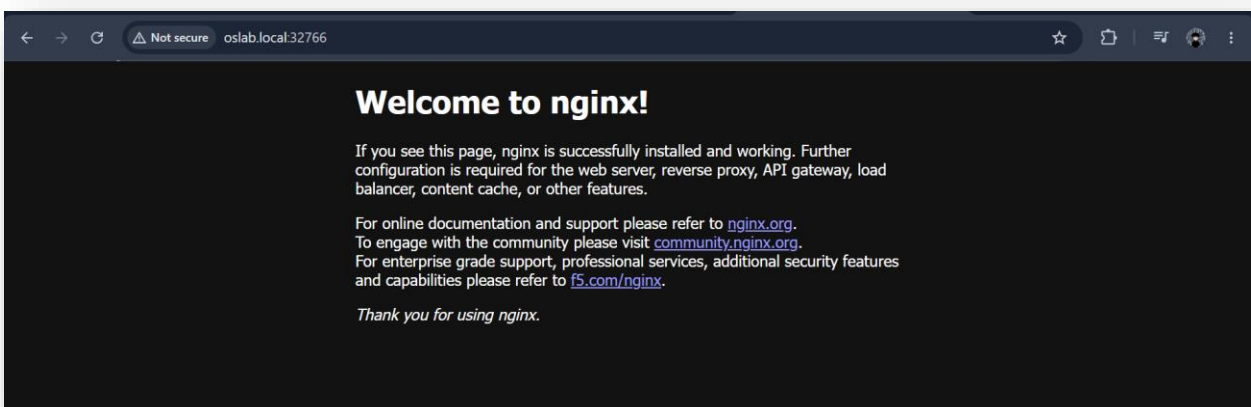


Phase 12: Initial Ingress Configuration

NGINX Ingress Controller was installed to manage external HTTP traffic for the Kubernetes cluster. Ingress provides a proper routing method by using a hostname instead of directly accessing the application through NodePort.

An Ingress rule was created for the hostname `oslab.local`. This rule forwards browser requests to the Kubernetes Service named `os-k8-service`, which then routes the traffic to the running application pods.

This was an initial Ingress test using the basic NGINX application. The final custom application Ingress was configured later using `portal.local`.





Phase 13: Custom Full Stack Application Development

A custom full stack web application was developed for the Kubernetes cluster. The application was designed with a modern blue Kubernetes theme and includes both frontend and backend functionality.

The frontend was created using HTML, CSS, and JavaScript. It contains a project overview section, team members section, and a message form. The backend was created using Node.js and Express.js. It handles API requests and connects the application with a MySQL database.

The main feature of this application is the message form. When a user submits a message, the data is sent to the backend and stored in MySQL. This feature will be used later to test persistence by deleting pods and checking whether the saved data remains available.

Files Created

File / Folder	Purpose
server.js	Backend server using Node.js and Express
package.json	Stores project dependencies and start script
Dockerfile	Used to build the custom application image
public/index.html	Main frontend page
public/style.css	Website styling and blue Kubernetes theme
public/script.js	Handles form submission and loads saved messages

Technologies Used

Technology	Purpose
HTML	Website structure
CSS	Website design and layout
JavaScript	Frontend interactivity
Node.js	Backend runtime
Express.js	Backend web framework
MySQL	Database for storing messages
Docker	Containerizing the application

```
ghost@k8-master:~/k8-os-lab-portal$ ls
Dockerfile  package.json  public  server.js
ghost@k8-master:~/k8-os-lab-portal$ ls public
index.html  script.js  style.css
```





Phase 14: Docker Image Build and Push

The custom full stack application was containerized using Docker. A Dockerfile was created to build the Node.js application image. The image includes the backend server, frontend files, and required dependencies.

After building the image, it was pushed to Docker Hub so that Kubernetes worker nodes could pull and run it inside the cluster.

```
ghost@k8-master:~/k8-os-lab-portal$ docker push ghost1027/k8-os-lab-portal:v1
The push refers to repository [docker.io/ghost1027/k8-os-lab-portal]
fff4e2c1b189: Mounted from library/node
9a1fa0b5c721: Pushed
6a0ac1617861: Mounted from library/node
4feea04c1543: Mounted from library/node
920728d32620: Pushed
b2cbbfe903b0: Mounted from library/node
ea0377b73910: Pushed
2d2d5c603d3a: Pushed
db4daa1551b0: Pushed
v1: digest: sha256:27148eb23e80cbc27067416226f08e1f9ed81fc8eb554763739cf687392696b7 size: 2275
ghost@k8-master:~/k8-os-lab-portal$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ghost1027/k8-os-lab-portal:v1	27148eb23e80	229MB	53.8MB	
ghostanimal1027/k8-os-lab-portal:v1	27148eb23e80	229MB	53.8MB	
node:20-alpine	fb4cd12c85ee	194MB	48.8MB	

Phase 15: Persistent Storage and MySQL Setup

A new namespace named `custom-app-lab` was created for the custom full stack application. Persistent storage was configured for MySQL using a `PersistentVolume` and `PersistentVolumeClaim`.

The MySQL pod was connected to the PVC so that database records remain saved even if the MySQL pod is deleted or restarted. A Kubernetes Secret was also created to store the database password and database name securely.

A ClusterIP Service named `mysql-service` was created so the custom Node.js application can connect to MySQL inside the cluster.





```
ghost@k8-master:~/k8-os-lab-portal$ kubectl get pods -n custom-app-lab -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE       NOMINATED NODE   READINESS GATES
custom-mysql-6bcc96ccf6-m6dbd       1/1     Running   0           14s   172.16.80.147   k8-worker1 <none>          <none>

ghost@k8-master:~/k8-os-lab-portal$ kubectl get svc -n custom-app-lab
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
mysql-service   ClusterIP   10.98.12.158 <none>        3306/TCP   26s
```

Phase 16: Custom Web Application Deployment

The custom full-stack web application was deployed inside the Kubernetes cluster in the `custom-app-lab` namespace. The application image was pulled from Docker Hub using the image name `ghost1027/k8-os-lab-portal:v2`.

The application was deployed using a Kubernetes Deployment with two replicas. This allows Kubernetes to run multiple copies of the web application and provides better availability. Both application pods were scheduled on `k8-worker2`.

The backend application connects to the MySQL database using the internal Kubernetes service `mysql-service`. Database credentials were provided using Kubernetes Secrets, which keeps sensitive values such as the database password separate from the application code.

A NodePort Service was created to expose the custom web application outside the cluster. This allowed the website to be accessed from the host machine browser.

Health checks were also configured using liveness and readiness probes. These probes allow Kubernetes to check whether the application is running correctly and ready to receive traffic.

```
ghost@k8-master:~/k8-os-lab-portal$ kubectl get pods -n custom-app-lab -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE       NOMINATED NODE   READINESS GATES
custom-mysql-6bcc96ccf6-m6dbd       1/1     Running   0           11m   172.16.80.147   k8-worker1 <none>          <none>
k8-os-lab-portal-8bd44b6d4-5k88n     1/1     Running   0           8m8s  172.16.86.147   k8-worker2 <none>          <none>
k8-os-lab-portal-8bd44b6d4-xbt88     1/1     Running   0           8m8s  172.16.86.146   k8-worker2 <none>          <none>
```

Phase 17: Persistence and Self-Healing Testing

State persistence and self-healing were tested using the custom full-stack application. A message was submitted through the web form and stored inside the MySQL database.

First, one of the custom application pods was manually deleted. Kubernetes automatically recreated the pod because the application was managed by a Deployment. After the new pod started, the website was refreshed and the saved message was still visible.





Next, the MySQL pod was manually deleted. Kubernetes recreated the database pod, and the saved message was still available because the database was connected to persistent storage using a PersistentVolumeClaim.

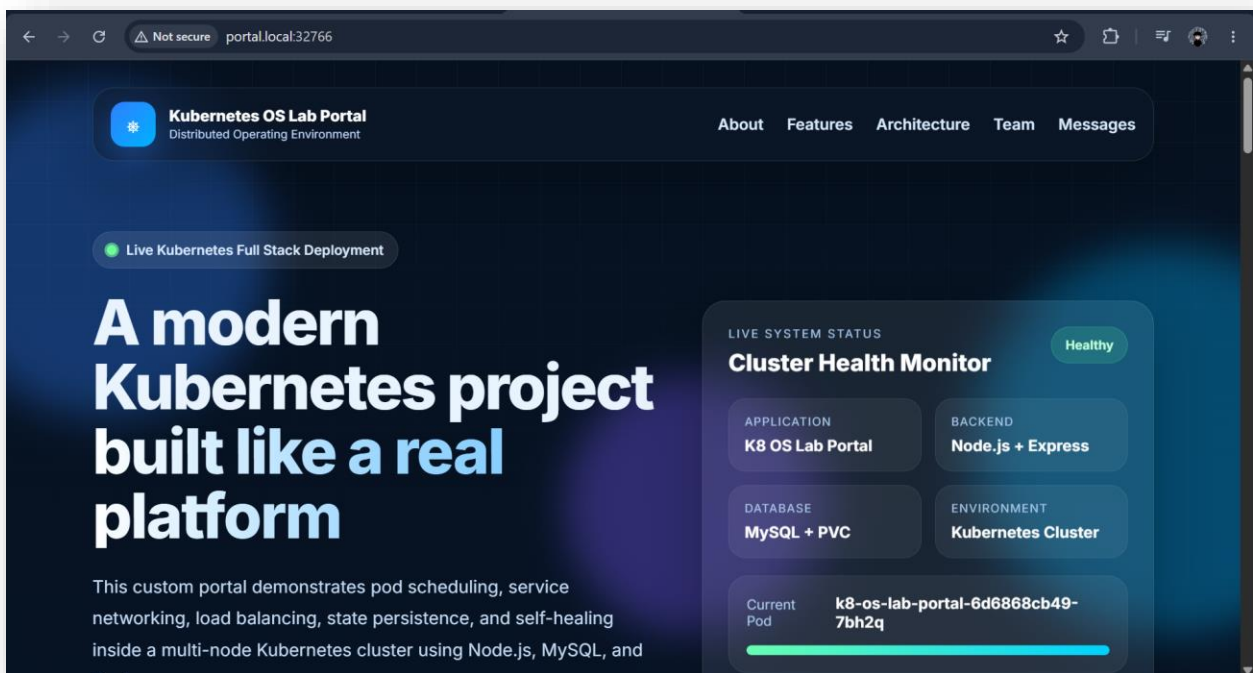
This confirmed that the application supports self-healing and that database records remain stored even after pod deletion or recreation.

Phase 18: Ingress Configuration for Custom Application

NGINX Ingress was configured for the custom full-stack application. Instead of accessing the website directly through the NodePort service, an Ingress rule was created to route traffic using the hostname portal.local.

The Ingress rule forwards external HTTP requests to the Kubernetes Service named k8-os-lab-service, which then sends traffic to the running application pods. This provides a cleaner and more realistic method of accessing the application.

The Windows hosts file was updated so that portal.local resolves to a worker node IP address.





Phase 19: Final Testing and Verification

Final testing was performed to verify that the complete Kubernetes cluster and custom full-stack application were working properly. All cluster nodes were checked and were in the `Ready` state.

The Kubernetes system pods, including Calico, CoreDNS, kube-proxy, API Server, Scheduler, Controller Manager, and etcd, were verified. This confirmed that networking, DNS, scheduling, and control plane services were functioning correctly.

The custom application pods and MySQL pod were checked inside the `custom-app-lab` namespace. The custom Node.js application was running with two replicas, while the MySQL database was running with persistent storage.

The application was accessed from the browser using NodePort and Ingress. The `/api/health` endpoint was also tested to confirm that the application was healthy and responding correctly.

Persistent storage was verified using `PersistentVolume` and `PersistentVolumeClaim` status. The saved messages remained available after pod deletion and recreation, confirming that data persistence was working successfully.

7. Conclusion

This project successfully demonstrated how Kubernetes can be used as a distributed operating environment for applying important Operating System concepts in a practical way. A multi-node Kubernetes cluster was created using one control plane node and two worker nodes on Ubuntu Server virtual machines.

The cluster was able to manage containerized applications, schedule pods across worker nodes, provide internal networking, expose services, and automatically recover failed pods. Features such as Deployments, Services, Ingress, RBAC, Kubernetes Dashboard, resource limits, health checks, and persistent storage were implemented to make the project close to a real-world environment.

A custom full-stack web application was developed using HTML, CSS, JavaScript, Node.js, Express.js, and MySQL. The application allowed users to submit messages through a modern web interface, and the data was stored inside the MySQL database running in the Kubernetes cluster.

Persistent storage was configured for MySQL using `PersistentVolume` and `PersistentVolumeClaim`. This ensured that database records were not lost even after deleting or restarting the MySQL pod. The self-healing test also showed that Kubernetes automatically recreated deleted application pods and maintained the desired state.

Overall, this project helped us understand OS concepts such as process management, scheduling, networking, resource allocation, access control, monitoring, fault tolerance, and storage





management in a distributed system. It also gave hands-on experience with modern container orchestration, Kubernetes administration, Docker image deployment, secure access control, and full-stack application hosting.

